UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 0. Real-bank slots: 11. Unofficial model answers:

   11/11.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (7).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The X machine is the Atanasoff–Berry Computer (ABC), the first automatic electronic digital computer to use the binary system. It was a special-purpose machine designed to solve systems of linear equations, using electronic arithmetic/logic units for addition and subtraction, a rotating drum memory holding thirty 50-bit binary numbers, and input/output mechanisms.

(b) The inventor of the X machine (ABC) was John Vincent Atanasoff, who designed and built it with his graduate assistant Clifford Berry at Iowa State College (now Iowa State University).

(c) Based on Figure 1, the input component is the card mechanism (punched-card reader/mechanism) used to enter data and instructions, and the output component is the output meter (a meter/display) used to read results. The figure labels include 'card mechanism' on the input side and 'meter' on the output side, together with the rotating drum memory and arithmetic/logic units.

Unofficial marking guide:

• 1 mark: (a) Correctly identifies X machine as the Atanasoff–Berry Computer (ABC) and/or states it is the first automatic electronic digital computer using binary.

• 2 marks: (b) Names John Vincent Atanasoff as inventor (1 mark); mentions Clifford Berry as co-builder/assistant or Iowa State College context (1 mark).

• 2 marks: (c) Identifies input component as card mechanism/punched-card reader (1 mark); identifies output component as meter/output meter (1 mark).

Model confidence: medium.

Model note: The question text contains OCR errors and the figure is only partially legible; 'X machine' is assumed to be the Atanasoff–Berry Computer (ABC) based on the description (binary, 30 ALUs, rotating drum memory of thirty 50-bit numbers). Figure 1 labels are interpreted conservatively: 'card mechanism' as input and 'meter' as output; if the figure shows additional I/O devices, those should be read directly from the figure.

QUESTION A2 (5 marks)

Topic: Computer generations and history

Source: CSC429 (10).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The machine X is the Analytical Engine, designed by Charles Babbage. (The text's clue that it 'solves four basic arithmetic functions' and 'improved its calculations design from Stepped Reckoner' points to Babbage's Analytical Engine; the Difference Engine was his earlier special-purpose machine.)

(b) The Analytical Engine is regarded as a first-generation (mechanical) computing device. It was designed in the 1830s, before the electronic generations, and belongs to the mechanical/pre-electronic era often labelled the first generation in historical surveys.

(c) Its function was to perform the four basic arithmetic operations (addition, subtraction, multiplication and division) automatically, and more generally to carry out general-purpose calculations under stored-program control using punched cards.

(d) The machine invented prior to machine X was the Stepped Reckoner (Leibniz's stepped reckoner), as stated in the snippet; the earlier mechanical calculator (Pascal's Pascaline) may also be accepted.

(e) Charles Babbage is given the title 'Father of the Computer' (father of computing) because of his success in designing the Analytical Engine.

Unofficial marking guide:

• 1 mark: (a) Analytical Engine

• 1 mark: (b) First generation (mechanical/pre-electronic era)

• 1 mark: (c) Performs the four basic arithmetic operations automatically / general-purpose calculation

• 1 mark: (d) Stepped Reckoner (accept Pascaline/early mechanical calculator)

• 1 mark: (e) Father of the Computer

Model confidence: high.

Model note: The extracted text does not explicitly name machine X; the identification as the Analytical Engine is inferred from the clues (four arithmetic functions, improved from the Stepped Reckoner, never completed, first machine deserving the name

© Hak Cipta Universiti Teknologi MARA

computer). If the figure labels the machine differently, read the figure. Generation classification is conventional; some syllabi place mechanical devices before the numbered generations.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Source: CSC429 (21).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A block is a group of data (or storage locations) that is transferred or handled as a single unit between the computer and its storage or I/O devices — for example, a fixed number of bytes read from or written to memory/disk in one operation. In the context of Figure 1, the whole string "CSC429 is awesome!" can be regarded as one block of characters stored contiguously.

(b) The smallest addressable unit is the byte (8 bits). Each character, including the space, occupies one byte and therefore has its own unique address; individual bits within a byte are not separately addressable.

(c) Counting the characters in Figure 1, "CSC429 is awesome!" = C,S,C,4,2,9 (6) + space (1) + i,s (2) + space (1) + a,w,e,s,o,m,e (7) + ! (1) = 18 characters.
(i) Byte: 18 characters × 1 byte/character = 18 bytes.
(ii) Bit: 18 bytes × 8 bits/byte = 144 bits.
(iii) Word: the system word is 2 bytes, so 18 bytes ÷ 2 bytes/word = 9 words.

Unofficial marking guide:

- *1 mark: (a) correct definition of a block as a group of data/storage locations handled as one unit*
- *1 mark: (b) byte stated as the smallest addressable unit*
- *1 mark: (c)(i) 18 bytes with correct character count*
- *1 mark: (c)(ii) 144 bits (18 × 8) with working*
- *1 mark: (c)(iii) 9 words (18 ÷ 2) with working*

Model confidence: high.

Model note: Assumes the string in Figure 1 is exactly 'CSC429 is awesome!' (18 characters including the single space and the exclamation mark); if the figure shows additional spaces or characters, recount accordingly. 'Word = two bytes' is taken from the question stem.

QUESTION A4 (5 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (12).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two main components of a Von Neumann Architecture: (1) the Central Processing Unit (CPU), comprising the control unit and the arithmetic-logic unit (ALU), and (2) the Main Memory (store), which holds both program instructions and data. Other acceptable components include the input/output (I/O) units and the buses (address, data and control) that interconnect them. Any two of these are sufficient.

(b) A general-purpose computer is a computer that is not dedicated to a single fixed task; it can be programmed to perform a wide variety of different tasks by loading different programs into its memory. Because instructions and data are stored together in the same memory (the stored-program concept), the same hardware can be reused for many applications (e.g. word processing, calculation, simulation) simply by changing the program, rather than by rewiring or rebuilding the machine.

(c) One of the earliest computers built on the Von Neumann Architecture: the EDSAC (Electronic Delay Storage Automatic Calculator, completed 1949 at Cambridge) — also acceptable are the Manchester Baby/SSEM (1948), the EDVAC, or the IAS machine. Naming any one of these earns the mark.

Unofficial marking guide:

- *1 mark: correctly names one main component of the Von Neumann Architecture (e.g. CPU, main memory, I/O, bus).*
- *1 mark: correctly names a second, distinct main component.*
- *1 mark: defines general-purpose computer as one that can be programmed to perform many different tasks / is not dedicated to a single fixed function.*
- *1 mark: explains that this is achieved by the stored-program concept — loading different programs into memory to reuse the same hardware.*
- *1 mark: names one valid early Von Neumann-based computer (e.g. EDSAC, Manchester Baby/SSEM, EDVAC, IAS machine).*

© Hak Cipta Universiti Teknologi MARA

Model confidence: high.
Model note: Question text is clear; no significant OCR correction needed. For (a) the marking accepts any two valid components since the question does not restrict to a specific pair. For (c) several early machines are historically valid; the guide accepts any one.

QUESTION A5 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (5).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The three parts of Figure 2 show the standard bus/interface families used to connect external devices to a computer. Typical labels are: (p) PCI Express (PCIe) — a high-speed serial point-to-point expansion bus used for graphics cards and other high-bandwidth adapters; (q) USB (Universal Serial Bus) — a serial interface for low-to-medium-speed peripherals such as keyboards, mice, printers and flash drives; (r) FireWire (IEEE 1394) — a serial interface for high-speed devices such as digital video cameras and external drives. (The exact label for each sub-figure must be confirmed from the connector/port drawing in Figure 2; the extracted text does not show the shapes.)

(b) A 'hot' connection (hot-plug / hot-swap capability) means a device can be connected to or disconnected from the running system without powering down or rebooting the computer, and the operating system detects and configures it automatically. This requires the interface and its controller to support safe insertion/removal (e.g. power and signal sequencing, surge protection) and driver support for dynamic configuration.

(c) The interfaces with hot connection capability are USB and FireWire (IEEE 1394); both are designed for hot-plugging. PCIe is an internal expansion bus and is not normally hot-plugged in a desktop (though some server/high-end implementations support hot-plug).

Unofficial marking guide:

- *2 marks: correct label for (p) with a brief justification*
- *2 marks: correct label for (q) with a brief justification*
- *2 marks: correct label for (r) with a brief justification*
- *2 marks: correct definition of 'hot' connection (connect/disconnect without powering down, automatic detection/configuration)*
- *1 mark: naming USB as hot-connectable*
- *1 mark: naming FireWire (IEEE 1394) as hot-connectable*

Model confidence: medium.
Model note: Figure 2 images are not available as extracted text, so the mapping of labels (p), (q), (r) to PCIe/USB/FireWire is assumed from the standard set of external-device interfaces; the actual connector drawings must be read from the figure to confirm which label belongs to which sub-figure. The question text has OCR/formatting errors (e.g. 'Figure 2 (p) - (r)' and repeated subpart labels) which were conservatively interpreted.

QUESTION A6 (10 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (14).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a)(i) MIMD with shared memory: Draw a single Memory Unit (MU) at the top, connected by a common bus/interconnection network to two or more Processing Units (PU1, PU2, ...). Each PU has its own Control Unit (CU1, CU2, ...) feeding it. Show one Instruction Stream (IS1, IS2, ...) entering each CU from the shared memory, and one Data Stream (DS1, DS2, ...) flowing between each PU and the shared MU. Label the common memory as 'Shared Memory (MU)' and the bus as 'Interconnection Network / Shared Bus'. Key point: all processors access the same global address space.

(a)(ii) MIMD with distributed memory: Draw two or more independent nodes, each node containing its own CU, PU and local MU. Show an Instruction Stream (IS1, IS2, ...) from each local MU to its CU, and a Data Stream (DS1, DS2, ...) between each PU and its local MU. Connect the nodes with an interconnection network (e.g., message-passing network) and label it 'Interconnection Network (message passing)'. Key point: each processor has a private local memory; no shared global memory.

(b)(i) Example of MIMD with shared memory: a multicore server/workstation (e.g., dual-socket Intel Xeon or AMD EPYC system) where all cores share the same main memory; alternatively a symmetric multiprocessor (SMP) such as a modern multi-core PC.

© Hak Cipta Universiti Teknologi MARA

(b)(ii) Example of MIMD with distributed memory: a computer cluster (e.g., a Beowulf cluster of PCs connected by Ethernet/InfiniBand) or a massively parallel processor (MPP) supercomputer; each node has its own memory and communicates by message passing (MPI).

(c) Advantage of MIMD with distributed memory: it is highly scalable — adding more nodes adds both processing power and local memory bandwidth, and there is no single shared-memory bus to become a bottleneck, so systems can grow to thousands of processors (e.g., clusters/MPPs) at relatively low cost using commodity nodes.

Unofficial marking guide:

• 2 marks: (a)(i) correct diagram of MIMD shared memory — one shared MU, multiple CU/PU pairs, common interconnection, IS and DS clearly labelled

• 2 marks: (a)(ii) correct diagram of MIMD distributed memory — multiple nodes each with own CU, PU and local MU, interconnection network, IS and DS clearly labelled

• 1 mark: (b)(i) valid example of MIMD shared memory (e.g., multicore/SMP server)

• 1 mark: (b)(ii) valid example of MIMD distributed memory (e.g., cluster/MPP)

• 2 marks: (c) one clear advantage of distributed-memory MIMD (e.g., scalability/no shared-memory bottleneck) with brief justification

Model confidence: high.

Model note: Figure 3 is only available as extracted text; the SISD baseline is assumed to be the standard CU–PU–MU arrangement with IS into CU and DS between PU and MU. Diagrams must be drawn by the candidate; the answer describes the required topology and labels. Examples given are illustrative — any correct shared-memory or distributed-memory MIMD system should be accepted.

QUESTION A7 (10 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (15).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Symmetric Multiprocessor (SMP) organization redrawn from Figure 2: In an SMP all processors are identical and have equal access to a single shared main memory through one common interconnection network (bus or switch). Redraw as: [Processor 1] [Processor 2] ... [Processor n] each connected by its own link to a single central Interconnection Network (shared bus/switch); the Interconnection Network connects to ONE shared Main Memory block (and typically shared I/O). Unlike the NUMA figure, there is no per-processor local memory and no separate Memory 1/Memory 2 — every processor sees the same memory with the same access time (UMA). Key labels: n identical processors, one shared interconnection network, one shared main memory, equal access latency for all processors.

Unofficial marking guide:

• 1 mark: correct SMP topology drawn — multiple identical processors all connected to one common interconnection network and one single shared main memory (no private/local memories).

• 1 mark: correct labelling showing symmetry/equal access (all processors share the same memory with uniform access time; contrast with the distributed Memory 1/Memory 2 of Figure 2).

• 2 marks: one correct example of a tightly coupled MIMD architecture (e.g. symmetric multiprocessor / SMP with shared memory).

• 2 marks: a second correct example (e.g. multicore processor, or NUMA/cluster-of-SMP with shared address space).

• 2 marks: one valid disadvantage of tightly coupled MIMD clearly explained (e.g. limited scalability — shared bus/memory becomes a bottleneck as processors increase).

• 2 marks: a second valid disadvantage clearly explained (e.g. high contention/synchronization overhead, or lack of fault tolerance — failure of shared resource/memory brings down the system).

Model confidence: high.

Model note: Figure 2 text is OCR-garbled; assumed it shows multiple processors each with local memory (Memory 1, Memory 2) linked by an interconnection network, i.e. a NUMA layout, which is the standard contrast for part (a). Part (a) is a drawing task, so the description of the diagram is given in words; the actual sketch must be produced by the candidate. Examples in (b) accepted as any standard tightly coupled MIMD forms (SMP, multicore, NUMA). This is an unofficial model answer, not a marking scheme.

QUESTION A8 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (18).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) A cache is a small, fast memory that holds recently or frequently accessed instructions and data from

© Hak Cipta Universiti Teknologi MARA

main memory. It exploits locality of reference (temporal and spatial) to reduce the average time to access memory. It sits between the CPU and main memory (often with multiple levels L1, L2, L3) and is typically built from SRAM.

(b) Without a cache, the CPU would have to fetch every instruction and data item directly from main memory (DRAM). Because DRAM is much slower than the CPU (often tens to hundreds of times slower), the processor would stall frequently, drastically reducing performance. The memory bus would also become a bottleneck, and the system would suffer from increased average memory access time, lower throughput, and poor utilization of the CPU's execution units. Overall, the computer would run much slower and be unable to keep the CPU busy.

(c) L1 cache is faster than L2 cache. L1 is the smallest and closest cache to the CPU core, often integrated on the same chip and operating at the CPU's clock speed. It has lower access latency (typically a few cycles) because of its proximity and small size. L2 is larger and farther from the CPU (though often still on-chip), and has higher access latency (tens of cycles). The trade-off is that L1 is smaller and more expensive per byte, while L2 is larger and slower but still faster than main memory. Therefore, L1 provides faster speed due to its smaller size, closer proximity, and lower latency.

(d) Two write policies to ensure cache-main memory consistency are:
1. Write-through: Every write to the cache is also written to main memory simultaneously. This keeps main memory always up-to-date, ensuring consistency, but generates more memory traffic and can slow down writes. Often combined with a write buffer to reduce stalls.
2. Write-back (or write-behind): Writes are made only to the cache. The modified cache block is marked dirty and is written to main memory only when it is evicted (replaced). This reduces memory traffic and is faster for repeated writes to the same block, but main memory can be temporarily inconsistent; consistency is maintained at eviction. Requires a dirty bit per block.

Unofficial marking guide:

• 1 mark: Definition of cache as small, fast memory holding recently/frequently accessed data/instructions.
• 1 mark: Mention of locality of reference or bridging speed gap between CPU and main memory.
• 1 mark: Explanation that without cache, CPU must access slower main memory directly, causing stalls.
• 1 mark: Consequence: increased average memory access time, reduced performance/throughput, or bus bottleneck.
• 1 mark: Identification of L1 as faster than L2.
• 1 mark: Justification: L1 is smaller, closer to CPU, lower latency (or L2 larger, farther, higher latency).
• 2 marks: Write-through policy explained (write to both cache and main memory; consistency maintained; more traffic).
• 2 marks: Write-back policy explained (write only to cache, dirty bit, write to memory on eviction; less traffic; temporary inconsistency).
Model confidence: high.

QUESTION A9 (10 marks)

Topic: Data representation and number systems

Source: CSC429 (13).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The question asks to calculate $FIFA_{16} - 664_{16}$ using two techniques. The operands are 16-bit hexadecimal numbers: $FIFA_{16}$ and 0664_{16} (padded to 16 bits). The result is expected in 16-bit representation.

(i) Sign and magnitude [5 marks]
In sign and magnitude, the most significant bit (bit 15) is the sign bit (0 for positive, 1 for negative) and the remaining 15 bits represent the magnitude. First, convert both operands to sign-and-magnitude 16-bit binary:
- $FIFA_{16}$ = 1111 1111 1111 1010_2. The sign bit is 1 (negative), magnitude = 111 1111 1111 1010_2 = $7FFA_{16}$ = 32762_{10}. So FIFA represents -32762_{10}.
- 0664_{16} = 0000 0110 0110 0100_2. The sign bit is 0 (positive), magnitude = 664_{16} = 1636_{10}. So 0664 represents $+1636_{10}$.
Now compute FIFA − 664 = (−32762) − (+1636) = -34398_{10}. The magnitude 34398_{10} is greater than the maximum 15-bit magnitude (32767), so it cannot be represented in 16-bit sign-and-magnitude.
Therefore, the operation overflows the sign-and-magnitude range. The correct mathematical result is

-34398_{10}, which would require at least 17 bits (sign + 16 magnitude bits). In 16-bit sign-and-magnitude, the result is not representable; overflow occurs.

(ii) 2's complement [5 marks]
In 2's complement, represent both operands in 16-bit 2's complement and perform subtraction as addition of the 2's complement of the subtrahend.
- $FIFA_{16}$ is already in 2's complement form: $FIFA_{16}$ = 1111 1111 1111 1010_2. Its decimal value is -6_{10} (since FIFA = −(0000 0000 0000 0110) = −6).
- 0664_{16} = 0000 0110 0110 0100_2 = $+1636_{10}$.
Compute FIFA − 664 = FIFA + (−664). Find the 2's complement of 0664_{16}: invert bits → $F99B_{16}$, add 1 → $F99C_{16}$. So -664_{16} = $F99C_{16}$.
Now add: $FIFA_{16}$ + $F99C_{16}$.
FIFA = 1111 1111 1111 1010
+ F99C = 1111 1001 1001 1100
= 1 1111 1001 1001 0110 (17 bits). Discard the carry out of bit 15: result = $F996_{16}$.
Check: $F996_{16}$ = 1111 1001 1001 0110_2. Since sign bit is 1, it is negative; magnitude = 2's complement of F996 = $066A_{16}$ = 1642_{10}. So result = -1642_{10}. Verify: −6 − 1636 = −1642, correct. No overflow because the true result −1642 is within the 16-bit 2's complement range (−32768 to 32767).

Unofficial marking guide:

• 1 mark: Correctly convert $FIFA_{16}$ to sign-and-magnitude 16-bit binary and identify sign and magnitude (or decimal equivalent −32762).
• 1 mark: Correctly convert 664_{16} to sign-and-magnitude 16-bit binary and identify sign and magnitude (or decimal equivalent +1636).
• 1 mark: Perform subtraction in sign-and-magnitude (or decimal) to obtain true result −34398.
• 1 mark: Recognize that −34398 exceeds the 15-bit magnitude range and state overflow/not representable in 16-bit sign-and-magnitude.
• 1 mark: Correct final conclusion for sign-and-magnitude (overflow, result not representable).
• 1 mark: Correctly represent $FIFA_{16}$ in 16-bit 2's complement (or state decimal −6).
• 1 mark: Correctly represent 664_{16} in 16-bit 2's complement (or state decimal +1636).
• 1 mark: Compute 2's complement of 664_{16} correctly ($F99C_{16}$).
• 1 mark: Perform 16-bit addition $FIFA_{16}$ + $F99C_{16}$ and discard final carry to get $F996_{16}$.
• 1 mark: Verify result $F996_{16}$ = -1642_{10} and state no overflow.
Model confidence: high.
Model note: OCR errors corrected: 'FIFAie' interpreted as $FIFA_{16}$, '664i6' as 664_{16}. Assumed 16-bit word length because $FIFA_{16}$ is a 16-bit hexadecimal number. In sign-and-magnitude, the result overflows; the question may expect showing the overflow condition. The 2's complement result is $F996_{16}$.

QUESTION A10 (10 marks)

Topic: Parallel and multiprocessor architecture
Source: CSC429 (4).pdf, page(s) 6
Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) One example for each classification of multiple processor organization. Multiple-processor organizations are commonly classified by how memory and processors are coupled: (i) Shared-memory multiprocessors (tightly coupled) — e.g. a Symmetric Multiprocessor (SMP) such as a dual/quad-core server where all CPUs share one main memory; (ii) Message-passing multicomputers (loosely coupled, distributed memory) — e.g. a cluster of workstations/PCs connected by a network; (iii) Non-Uniform Memory Access (NUMA) multiprocessors — e.g. a large server built from several SMP nodes each with local memory, connected by an interconnection network; (iv) Vector/SIMD or special-purpose parallel processors — e.g. a GPU or a vector supercomputer (Cray-style) used for data-parallel work. (Any one concrete example per classification earns the mark; the four classifications above cover the standard taxonomy.)

(b) Simplified architectures (described so they can be drawn):

i. Symmetric Multiprocessors (SMP): Draw N identical processors (P1…Pn) each connected through a common system bus (or crossbar/switch) to one shared main memory and to shared I/O. All processors have equal access to all memory and I/O; one OS manages the whole system. Sketch: [P1][P2]…[Pn] → common bus → [Shared Memory] and [I/O devices].

ii. Clusters: Draw several independent complete computers (each = CPU + local memory + local I/O running its own OS), each attached via a network interface to a high-speed interconnection

© Hak Cipta Universiti Teknologi MARA

network/switch. No shared global memory; nodes communicate by message passing. Sketch: [Node1: CPU+Mem] [Node2: CPU+Mem] … [NodeN: CPU+Mem] each → NIC → [Interconnect/Switch].

iii. NUMA: Draw several SMP nodes, each node containing processors plus its own local memory (and possibly local I/O), all nodes joined by an interconnection network. A processor accesses its local memory fast and remote nodes' memory more slowly (non-uniform access time). Sketch: [Node A: P+P+Local Mem] — [Interconnect Network] — [Node B: P+P+Local Mem] — … with each node's memory attached locally.

Unofficial marking guide:

• 1 mark: correct example for shared-memory multiprocessor (e.g. SMP server)

• 1 mark: correct example for message-passing multicomputer/cluster

• 1 mark: correct example for NUMA multiprocessor

• 1 mark: correct example for a fourth classification (vector/SIMD/GPU or equivalent)

• 2 marks: SMP diagram showing multiple processors sharing one memory via a common bus/switch

• 2 marks: cluster diagram showing independent computers (CPU+local memory) linked by a network, no shared memory

• 2 marks: NUMA diagram showing multiple nodes each with local memory joined by an interconnect, with local vs remote access distinction

Model confidence: medium.

Model note: The question does not name a specific classification scheme; the answer uses the standard shared-memory / message-passing / NUMA / vector-SIMD taxonomy. If the lecturer's notes use a different set of classifications, examples should be matched to those. Diagrams are described in text because only extracted question text is available.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (20).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 1's extracted text reads z=A2-B*C+jl_ D+5, which is garbled. The most conservative reading consistent with the instruction set (which has ADD, SUB, MUL, DIV but no square-root or exponent instruction) is z = (A² − B·C) / (D + 5). The jl_ is taken as a division bar /, A2 as A² (computed as A·A), and D+5 as the denominator. All three programs below compute this expression and leave the result in z. (If the intended equation differs, the same structure applies; see notes.)

Three-address program (each instruction names two sources and one destination; A, B, C, D are fixed registers, x, y, z are temporaries):

```
MUL (A, A, x)     ; x = A * A
MUL (B, C, y)     ; y = B * C
SUB (x, y, x)     ; x = A^2 - B*C   (numerator)
ADD (D, 5, y)     ; y = D + 5       (denominator)
DIV (x, y, z)     ; z = (A^2 - B*C) / (D + 5)
```

The three-address form is the most compact: one instruction per operation, no explicit data movement, because operands and result are all named in the instruction.

One-address program (an implicit accumulator; LOAD/STOR move between memory/registers and the accumulator, ADD p etc. do ACC ← ACC op p):

```
LOAD A          ; ACC = A
MUL A           ; ACC = A * A
STOR x          ; x = A^2
LOAD B          ; ACC = B
MUL C           ; ACC = B * C
STOR y          ; y = B*C
LOAD x          ; ACC = A^2
SUB y           ; ACC = A^2 - B*C
STOR x          ; x = numerator
LOAD D          ; ACC = D
ADD 5           ; ACC = D + 5
STOR y          ; y = denominator
LOAD x          ; ACC = numerator
DIV y           ; ACC = numerator / denominator
STOR z          ; z = final answer
```

The one-address form needs explicit LOAD/STOR because only one operand (the accumulator) is implicit; intermediate results must be spilled to x and y.

Zero-address program (a stack machine; PUSH p pushes operand p, POP p pops the top into p, and ADD/SUB/MUL/DIV pop two operands and push the result):

```
PUSH A          ; stack: A
PUSH A          ; stack: A, A
MUL             ; stack: A*A
PUSH B          ; stack: A*A, B
PUSH C          ; stack: A*A, B, C
MUL             ; stack: A*A, B*C
SUB             ; stack: A*A - B*C
PUSH D          ; stack: num, D
PUSH 5          ; stack: num, D, 5
ADD             ; stack: num, D+5
DIV             ; stack: num / (D+5)
POP z           ; z = final answer
```

The zero-address form uses the stack for all operands and results; note the operand order (push

© Hak Cipta Universiti Teknologi MARA

numerator first, then denominator) so that DIV computes numerator ÷ denominator.

Summary: all three programs compute z = (A² − B·C)/(D + 5) and store the final result in z; they differ only in how operands are addressed (explicitly named, via an accumulator, or via a stack).

Unofficial marking guide:

• 2 marks: correct interpretation/statement of the equation to be computed (A² − B·C)/(D + 5), including A² as A·A and D+5 as denominator

• 6 marks: correct three-address program — 1 mark each for computing A·A, B·C, the subtraction (numerator), D+5 (denominator), the division, and storing the result in z

• 6 marks: correct one-address program — 1 mark each for correct use of LOAD/STOR to form A², B·C, numerator, denominator, division, and final STOR z (accumulator semantics correct)

• 6 marks: correct zero-address program — 1 mark each for correct PUSH sequence for A², correct PUSH/MUL for B·C, correct SUB for numerator, correct PUSH/ADD for D+5, correct DIV with operands in the right order, and POP z for the final result

Model confidence: medium.

*Model note: The equation in Figure 1 is OCR-garbled ('z=A2-B*C+jl_ D+5'). I conservatively read it as z = (A² − B·C)/(D + 5), since the instruction set has no exponent or square-root operation and A² must be computed as A·A. If the actual figure shows a different expression (e.g. a different denominator or an added term), the same three program structures apply but the exact instruction sequences and mark allocation per step would change. The register names A, B, C, D are treated as fixed registers and x, y, z as temporaries, with z holding the final answer, per the question. This is an unofficial model answer, not a marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA